

Theory:

1) Implicit conversion: Javascript automatically converts one data type into another when required.

Explicit conversion: The programmer manually converts one data type into another using function such as `Number()`, `string()`, or `Boolean()`

2) `Number("123")` // 123

`Number("123abc")` // NaN

"123" is completely numeric, so it converts successfully.
"123abc" contains non-numeric characters, so `Number()` returns NaN.

3) Both return "123"

The difference is that `string()` can safely convert null and undefined:

`string(null)` → "null"

`string(undefined)` → "undefined"

But calling `string()` directly on null or undefined causes an error.

4) `Boolean()` converts a value into either true or false.

The falsy values are:

false

0

-0

""

null

undefined

NaN

All other normal Javascript values are truthy.

5) Number() tries to convert the entire string into a number.

Number("123") // 123

Number("123abc") // NaN

parseFloat() reads an integer from the beginning and stops when it encounters an invalid character.

parseFloat("123abc") // 123

6) 4.2. parseFloat() reads the valid integer 4.2 from the beginning of the string and stops when it reaches p. It doesn't throw an error because 4.2 is a valid integer that could be parsed.

7) == is loose equality. It can perform type conversion before comparing.

=== is strict equality. It compares both the value and the type without performing type coercion.

Eg) 5 == "5" // true

5 === "5" // false

8) With ==, JavaScript converts the string "5" into the number 5. Therefore: 5 == 5 → true. With ===, the types must also be the same. "5" string, 5 number. Therefore: "5" === 5 → false.

9) A truthy value behaves like true when used in a Boolean context.

A falsy value behaves like false

Eg): Boolean("Hello") // truthy

Boolean("") // falsy

Boolean(0) // falsy

Boolean(10) // truthy

This matters because if conditions automatically evaluate values as Boolean.

10) 3.14

`parseFloat()` reads the valid floating point number from the beginning and stops when it reaches the text.

Output prediction:

11) 123

12) NaN

13) 0

14) 0

15) NaN

16) null

17) False

18) true

19) true

20) False

Error Finding:

21) O/P: NaN

Explanation: "abc" cannot be converted into a valid number.

So `Number()` returns the special value NaN. It is not a JavaScript error.

22) O/P: 11

Explanation: Because one operand is string, + operator performs string concatenation.

23) 4

Explanation : The `-` operator forces numeric conversion, so `"5"` becomes `5`.

24) O/P : true

Explanation : `==` has a special rule that considers null and undefined equal.

25) O/P : False

Explanation : `===` performs strict comparison, null and undefined are different values/types.

26) O/P : true

Explanation : Through Javascript type coercion rules, the empty array is converted toward an empty string and then to `0`, while `false` converts to `0`. Therefore the comparison becomes effectively `0===0`.

27) O/P : NaN

Explanation : There is no numeric value in the empty string for `parseInt()` to parse.

28) O/P : truthy

Explanation : `"0"` is a non empty string, so it is truthy.

`0` $\rightarrow$ falsey, `"0"` $\rightarrow$ truthy.

29) O/P : false

Explanation : NaN is a special value that is not equal to itself. To check for NaN use.

`Number.isNaN(value)`.

30) O/P: 3.14

Explanation: Unary + converts the string "3.14" into the number 3.14.

Coding:

31) let value = "42";

let number1 = Number(value);

O/P:

let number2 = Number(value, 10);

42

console.log(number1);

42

console.log(number2);

32) function toBoolean(value){

return Boolean(value);

}

console.log(toBoolean("Hello"));

O/P:

console.log(toBoolean(""));

true

console.log(toBoolean(0));

false

console.log(toBoolean(10));

false

true

33) let input = "19.99";

let price = parseFloat(input);

console.log(price);

O/P:

console.log(price * 2);

19.99

39.98

34)

let falsy values = [

false,

0,

-0,

0n,

""

null,
undefined,
NaN

```
};  
falseyValues.forEach(function (value) {  
  console.log(value, Boolean(value));  
});
```

All the listed values produce: false.

35) let a = "5";
let b = 5;

```
console.log(a == b);  
console.log(a === b);
```

O/P: true
false

36) function isValidNumber(value) {

```
  let number = Number(value);
```

```
  return !Number.isNaN(number);
```

```
}
```

```
console.log(isValidNumber("123"));
```

```
console.log(isValidNumber("abc"));
```

O/P: true
false

37) let binary = "1010";

```
let decimal = parseInt(binary, 2);
```

```
console.log(decimal);
```

O/P: 10

38) let number = 25;

```
let text = String(number);
```

```
console.log(`The number is ${text}`);
```

O/P:

The number is 25.

39) function convertToNumber(value){

let number = Number(value);

if (Number.isNaN(number)){

return 0;

}

return number;

}

console.log(convertToNumber("25"));

console.log(convertToNumber(50));

console.log(convertToNumber(true));

console.log(convertToNumber("abc"));

O/P:

25

50

true

abc

40)

console.log(Number("10"));

console.log(parseInt("10abc", 10));

O/P:

10

10

Real-World Scenarios:

41) Because from input values are string. If they are not converted, operators such as + may perform string concatenation instead of numeric addition.

For eg): "5" + 10 $\Rightarrow$ "510" After conversion: 5 + 10 $\Rightarrow$ 15

Therefore numeric input should be converted before performing calculations.

42)

Number()

Number("100") $\Rightarrow$ 100; Number() handles leading and trailing whitespace during numeric conversion.

43) Because == performs type coercion.

For eg)

"200" == 200 → true

Even though one value is string and the other is a number. This can allow an unexpected data type to pass validation. `===` is safer because it checks both value and type.

44) Use `Boolean()` to ensure a strict Boolean result.

For example, Conceptually:

`Boolean(Value)`

returns only:

or
true
false

For an actual checkbox, its checked property is already Boolean.

45) Use `parseInt()`.

It extracts the integer from the beginning:

`parseInt("25 years", 10) → 25`

`Number("25 years")` would return NaN because the complete string isn't numeric.

46) Because "false" is a non-empty string, so it is truthy.

Therefore:

`Boolean("false") → true`

If the stored value is the string "false", compare it with the expected string.

`stored value === "true"`

For example:

"false" == "true" → false.

This correctly interprets the stored text.

47) The discount is a string

if:

subtotal = 100

discount = "10"

then $100 + "10" = 10010$. This is string concatenation.

The discount should first be converted to a Number.

48)

Because 0 is a falsy value. Therefore, if (0) does not execute. If 0 is a valid quantity, don't use truthiness to determine whether the quantity is valid. Instead, use an explicit condition appropriate to the requirement, such as checking whether the value is null or undefined.

49) Identify which columns should contain numbers and convert only those columns to numbers.

General strategy:

- 1) Identify the expected data type.
- 2) Convert the string using the appropriate conversion method.
- 3) Validate the result.
- 4) Handle invalid values appropriately.

For numeric columns, Number() is often appropriate when the whole field must be numeric.

50) Use Number() and check the results with Number.isNaN().

Conceptually:

Convert the value $\rightarrow$ check Number.isNaN() $\rightarrow$
reject if true

For example:

Number("123") $\rightarrow$ 123

Number("abc") $\rightarrow$ NaN

So "abc" should be rejected as invalid rather than as a numeric value.